

ELEC6236 Design Report Form 2024/25

Email	mkb1u24@soton.ac.uk	ID Number (8 digits)	36641472
-------	--	-------------------------	----------

Achievement What did you achieve? Comment on your scores from the online code checker. (2 marks)

An Adder was designed to perform additions between two 16-bit floating point numbers. The design was simulated using the given testbench and with a variety of inputs and was synthesised to determine hardware utilisation and logic gate implementation.

The tests included cases such as the addition of 0 & 0, 1 & 0, and 1 & -1, which were executed with an error percentage of 0.00%. Tests 5 through 7 involved additions between rational numbers with different signs and exponents. In these calculations, a 0.00% error rate was achieved. However, since the sum results produced round offs, the number of significant figures can get reduced due to final normalisation step and the 16-bit limitation of the data type. The final test involved the addition of a negative Non-A-Number (NaN) and 1, which resulted in negative Infinity, with of all exponent bits set to 1. The design accounted for special input cases and this test verified the response of the design for such scenarios.

Technical Approach How did you implement the design? What functionality did you implement? (3 marks)

The Adder is implemented as a Synchronous implementation, using Sequential logic utilising non-blocking statement and employing an always_ff block sensitive to the positive edge of the clock. A State Machine is defined with in the block to sequentially execute the following steps: storing inputs, component extraction, checking for special cases, performing addition or subtraction based on the signs, sorting or shifting of output before finally storing the result and asserting the ready signal.

Detailed Functionality:

Extraction & Special Cases: The inputs are split into respective sign bit, biased exponent bits, and mantissa with leading 1 restored. Once extracted, special cases (e.g., zeros, infinity, and Not-a-Number) are detected. If such a case is found, the state machine jumps directly to the result state and concludes. For valid inputs, the exponents are compared and the mantissa with the restored 1 is shifted accordingly. The mantissa with the larger exponent is stored in the non-shift register and vice versa.

Alignment & Sum/Difference Operation: The exponent of the result is determined at this stage (it may be incremented or decremented in the final normalisation step as per the location of the most significant 1 of the sum/difference register). The sign bit of the result is also assigned according to the sign of the higher exponent or the larger mantissa if the exponents are equal. Once both the numbers are equivalent in terms of the exponents, the shifted and non-shifted mantissa are then added or subtracted based on the sign bits. The result is stored in a 9 bit register to handle a potential overflow due to addition of MSB 1s.

Normalisation & Result: Once the mathematic operation between the shifted and non-shifted mantissas is computed, the most significant 1 is located, and the bits following this 1 are extracted and saved as the resultant mantissa. This is done to comply with the 754 standard as the leading 1 is assumed and not part of the result. Due to the 9-bit limitation, a rounding to the nearest smaller number may occur depending on to leading 1 bit location. The exponent of the result is adjusted either by incrementing or decrementing with respect

to the bit position of this leading 1. The resultant sign bit, exponent bits and mantissa bits are finally concatenated in the sum register and is output through the bus, while simultaneously asserting the ready output to indicate start of the next addition sequence.

Testing and Evaluation How did you test and verify your design? How did you use the design software (ModelSim, Quartus and the online checker)? (3 marks)

To test and verify the working of the adder, Modelsim was used to compile and simulate the Adder code and the test bench. Upon compiling, syntax and compilation errors identified were corrected. To verify the complete logic, the test bench provided with the assignment was utilised. **\$display** and **\$monitor** constructs were used in the main module to monitor intermediate values and aid in debugging of the code.

The inputs in the test bench were manually changed for each run and then converted to the bfloat32 format using shortrealtobits. To test multiple values, the test bench was modified to include random numbers using the **min + \$random () % (max – min)** and perform multiple tests with the **\$repeat ()** construct. The test bench then simulated on Modelsim and the results of the designed bfloat16 adder were compared with the actual sum of the numbers. The simulation waveforms were used to verify the availability of the result and the assertion of the **ready** signal which was implemented by introducing a separate state in the final code to indicate when the design expects the next input in the subsequent clock cycle.

Quartus was then used to compile the design. DE1-SoC development board was selected, and the “analysis and synthesis” of the FPGA was performed. After synthesis, the compilation was carried out, including the placement and routing. The compilation report calculated the FPGA resource utilisation. The summary report indicated that logic utilisation was less than 1% (196/32070), with a total of 112 registers and 51 pins out of the total 457 pins used. Optimisation during the synthesis resulted in a merger between the mantissa b register and the restored mantissa b register, leading to a decrement of 42 registers. The RTL view generated using the Netlist viewer tool of Quartus, showed the logic implementation. It depicted the number of adders, multipliers and logic gates required for the design implementation.

Reflection What did you learn from this exercise? What would you do differently if you were to do the exercise again, and why? (2 marks)

Learning: Designing and simulating the adder highlighted the differences between blocking and non-blocking assignments and the combinational versus sequential behaviour of **always_comb** and **always_ff** blocks, particularly through the output waveforms. Another key insight was the rounding off due to mantissa limitation and the role of ready signal in indicating result availability and initiating the next operation.

Possible Improvements: To produce more accurate results, the register used for the Sum/Difference operation could be expanded from 9 bits to cater the rounding off. This way when the leading 1 is located and the remaining 7 bits are extracted, the round off takes place based on the next right bit such that can be added instead of padding with zeros which currently performs a round off similar to the ceiling function. (e.g. sum = 100000000 {+expanded 100000000}; if the highlighted bit is 1, it is added to the 9-bit sum).

The designed adder makes use of non-blocking statements only, for both the state changes and computations. The use of combination logic and blocking assignments could be advantageous for the state changes, as currently the input b does not follow the first input and is immediately received with input a. Alternatively, an @ (posedge clock) could be used in the existing design to delay receiving of input b.